Reflexión personal — Laboratorio 12: Agente Cloud

1. **¿Qué diferencia fundamental existe entre un CLI Agent y un agente cloud?**

La diferencia principal está en dónde y cómo trabaja. El CLI Agent (como Codex o Qwen) vive en mi terminal. Tengo que estar frente a la computadora, dándole instrucciones, viendo qué comandos ejecuta en tiempo real y aprobando cada paso. Es interactivo y síncrono.

El agente cloud (como Copilot Cloud Agent) es todo lo contrario. Vive en la nube. Yo le doy una tarea (un issue) y él se va a un entorno remoto a trabajar por su cuenta. No necesito estar frente a la computadora. Cuando termina, me devuelve un Pull Request listo para revisar. Es asíncrono y delegable.

2. **¿Qué parte del trabajo pudo continuar, aunque usted no estuviera ejecutando nada en su computadora?**

En el laboratorio, cuando asigné el issue a Copilot, él se fue a la nube, leyó los archivos, modificó `tasks.py`, actualizó las pruebas y las ejecutó. Todo eso pasó sin que yo tocara mi teclado. Incluso podía haber apagado mi computadora y el agente habría seguido trabajando. Eso es lo poderoso del agente cloud: el trabajo pesado ocurre en un servidor remoto, no en mi máquina.

3. **¿Por qué un Pull Request es una buena frontera de control entre el agente y el desarrollador?**

Porque el PR es una **propuesta**, no una imposición. El agente no puede modificar la rama `main` directamente. Primero tiene que crear una rama aparte y abrir un PR. Ahí yo puedo revisar el diff, ver qué archivos tocó, ejecutar las pruebas localmente y decidir si lo apruebo o no. Si el agente se equivoca o se sale del alcance, simplemente rechazo el PR. El PR me da el control final sobre lo que entra al proyecto.

4. **¿Qué riesgo existiría si el agente pudiera hacer merge automáticamente sin revisión?**

Sería un riesgo enorme. El agente podría introducir un bug, borrar código importante o modificar archivos que no debía. Aunque el agente diga "las pruebas pasaron", eso no garantiza que el cambio sea correcto o seguro. En el laboratorio lo vimos: el agente hizo un buen trabajo, pero si no hubiera revisado el diff y validado localmente, no habría tenido la certeza de que todo estaba bien. El merge automático eliminaría la supervisión humana, que es la última línea de defensa.

5. **¿Qué información debe contener un issue para que una tarea sea realmente delegable?**

Para que un agente (o un humano) pueda hacer la tarea sin preguntarme nada, el issue debe tener:

- **Objetivo:** qué debe quedar funcionando.
- **Alcance:** qué archivos puede tocar.
- **Restricciones:** qué no debe hacer (no agregar dependencias, no modificar ciertos archivos).
- **Criterios de aceptación:** casos concretos que deben pasar.
- **Validación:** cómo comprobar que está bien (comando de pruebas).

En el laboratorio, el issue que creé tenía todo eso, y el agente pudo implementarlo sin ambigüedades. Si el issue hubiera sido vago, el agente habría hecho cualquier cosa.

6. **¿En qué tipo de tareas usaría un agente cloud y en cuáles preferiría trabajar de forma interactiva?**

Usaría un agente cloud para tareas repetitivas o bien definidas: agregar una función con pruebas, corregir un bug concreto, refactorizar un módulo pequeño, actualizar la documentación. Son tareas que puedo delegar y revisar después.

Preferiría trabajar de forma interactiva (con un CLI Agent o directamente en VS Code) para tareas de exploración, diseño de arquitectura, o cuando necesito probar diferentes enfoques y tomar decisiones sobre la marcha. También para tareas que requieren mucha iteración o donde el agente podría desviarse fácilmente.